PYTHON PROGRAMMING

ASSIGNMENT 2 – CO2

Revised and completed solutions for Tasks 1 to 7

Name	G. Anushka
Register Number	1912260668
Course	CSA0825 – Python Programming
Slot	A
GitHub	https://github.com/anushkapearl21-jpg/CSA0825-co2.git

Note: This revised version includes executable Python 3 programs, representative test cases, sample outputs, and concise analysis for every task. Timings in Task 4 are measured when the program is run and may vary by computer.

TASK 1 – Decisions with Many Boundaries

Aim

To evaluate electricity subsidy eligibility and compare independent **if** statements with an **if–elif–else** decision structure.

Concept / Theory

Independent **if** statements are evaluated separately, so more than one matching condition can append more than one result. An **if–elif–else** chain selects one mutually exclusive path. The conditions below define the slabs: up to 100 units receives full subsidy; 101–200 units requires a ration card for partial subsidy; above 200 units requires senior-citizen status for special subsidy.

Procedure / Algorithm

1. Define two functions, one using separate if statements and one using if–elif–else.

2. Test eight households covering slab boundaries and eligibility combinations.

3. Print both results and compare whether each function returns one decision.

Python Program

```
def subsidy_separate_if(units, ration_card, senior):
    results = []
    if units <= 100:
        results.append("Full Subsidy")
    if 101 <= units <= 200 and ration_card:
        results.append("Partial Subsidy")
    if 101 <= units <= 200 and not ration_card:
        results.append("No Subsidy")
    if units > 200 and senior:
        results.append("Special Subsidy (Senior)")
    if units > 200 and not senior:
        results.append("No Subsidy")
    return results


def subsidy_if_elif(units, ration_card, senior):
    if units <= 100:
        return "Full Subsidy"
    elif units <= 200:
        if ration_card:
            return "Partial Subsidy"
        return "No Subsidy"
    else:
        if senior:
            return "Special Subsidy (Senior)"
        return "No Subsidy"


households = [
    (80, False, False), (100, True, False),
    (101, True, False), (150, False, False),
    (200, True, False), (201, False, True),
    (250, False, False), (300, True, True)
]

for h in households:
    units, card, senior = h
    print(h, "=>", subsidy_separate_if(*h),
          "| if-elif:", subsidy_if_elif(*h))
```

Sample Output

```
(80, False, False) => ['Full Subsidy'] | if-elif: Full Subsidy
(100, True, False) => ['Full Subsidy'] | if-elif: Full Subsidy
(101, True, False) => ['Partial Subsidy'] | if-elif: Partial Subsidy
(150, False, False) => ['No Subsidy'] | if-elif: No Subsidy
(200, True, False) => ['Partial Subsidy'] | if-elif: Partial Subsidy
(201, False, True) => ['Special Subsidy (Senior)'] | if-elif: Special Subsidy (Senior)
(250, False, False) => ['No Subsidy'] | if-elif: No Subsidy
(300, True, True) => ['Special Subsidy (Senior)'] | if-elif: Special Subsidy (Senior)
```

Analysis / Discussion

Both implementations produce the same single outcome for the defined non-overlapping slabs. Separate if statements are useful when conditions are independent; the if–elif–else structure makes exclusivity explicit. Each evaluation takes O(1) time and O(1) auxiliary space.

Result

The electricity subsidy decision logic was implemented and tested at lower, upper, and above-boundary values. The if–elif–else version clearly guarantees one decision.

TASK 2 – Loops that Count Attempts

Aim

To implement OTP verification for a bank kiosk using while and for loops, ignoring blank entries and locking after three incorrect non-blank attempts.

Concept / Theory

A while loop is suitable when the number of input events is not known in advance. Blank input is checked before incrementing the attempt counter, so it does not consume an attempt. A for loop can also implement the same limit using a bounded range and break.

Procedure / Algorithm

1. Set the correct OTP to 1234 and the maximum number of failed attempts to three.

2. Read input, trim whitespace, and ignore blank entries.

3. Stop on a correct OTP; otherwise increment the failed-attempt count.

4. Display a lockout message after three failed attempts.

Python Program

```
def verify_otp_while(entries):
    correct_otp = "1234"
    attempts = 0
    for user_input in entries:  # supplied inputs simulate kiosk entries
        user_input = user_input.strip()
        if user_input == "":
            print("Blank entry detected. Not counted.")
            continue
        if user_input == correct_otp:
            print("OTP Verified Successfully!")
            return
        attempts += 1
        print("Incorrect OTP. Attempts remaining:", 3 - attempts)
        if attempts == 3:
            print("Account LOCKED for 30 minutes.")
            return
    print("Input ended before verification or lockout.")


def verify_otp_for(entries):
    correct_otp = "1234"
    attempts = 0
    for user_input in entries:
        user_input = user_input.strip()
        if user_input == "":
            print("Blank entry detected. Not counted.")
            continue
        if user_input == correct_otp:
            print("OTP Verified Successfully!")
            break
        attempts += 1
        print("Incorrect OTP. Attempts remaining:", 3 - attempts)
        if attempts == 3:
            print("Account LOCKED for 30 minutes.")
            break


print("WHILE LOOP:")
verify_otp_while(["", "1111", "2222", "1234"])
print("\nFOR LOOP:")
verify_otp_for(["1111", "", "2222", "3333"])
```

Sample Output

```
WHILE LOOP:
Blank entry detected. Not counted.
Incorrect OTP. Attempts remaining: 2
Incorrect OTP. Attempts remaining: 1
OTP Verified Successfully!

FOR LOOP:
```

```
Incorrect OTP. Attempts remaining: 2
Blank entry detected. Not counted.
Incorrect OTP. Attempts remaining: 1
Incorrect OTP. Attempts remaining: 0
Account LOCKED for 30 minutes.
```

Analysis / Discussion

The attempt counter increases only for non-blank incorrect entries. The while version returns immediately after success or lockout; the for version uses break for the same outcomes. Both use O(1) auxiliary space. A recursive retry design would use O(a) call-stack space for a attempts.

Result

OTP verification correctly ignores blank entries, accepts the correct OTP, and locks the account after three failed non-blank attempts. The input list is used to make the sample output reproducible; replace it with input() for an interactive kiosk.

TASK 3 – Engineer a Vehicle Registration Number Validator

Aim

To normalize and validate a vehicle registration number using string methods and slicing, without regular expressions.

Concept / Theory

Python strings are immutable. Methods such as upper() and replace() return a new string. Slicing extracts the state, district, series, and final four-digit number. The accepted normalized lengths are nine or ten characters, matching one- or two-letter series formats.

Procedure / Algorithm

1. Remove spaces and hyphens and convert the input to uppercase.

2. Check that the normalized length is 9 or 10.

3. Extract state, district, series, and the last four digits using slicing.

4. Validate alphabetic and numeric sections and format valid plates.

Python Program

```
def validate_and_format_plate(raw_plate):
    clean = raw_plate.replace("-", "").replace(" ", "").upper()

    if len(clean) not in (9, 10):
        return False, "Invalid Length"

    state = clean[:2]
    district = clean[2:4]
    series = clean[4:-4]
    number = clean[-4:]

    if (state.isalpha() and district.isdigit()
            and series.isalpha() and number.isdigit()):
        return True, f"{state} {district} {series} {number}"

    return False, "Invalid Format"


test_plates = [
    "tn09a1234", "TN 09 AB 1234",
    "TN-09-AB-1234", "TN09A12B4",
    "T109AB1234", "TN091234"
]

for plate in test_plates:
    print(plate, "=>", validate_and_format_plate(plate))
```

Sample Output

```
tn09a1234 => (True, 'TN 09 A 1234')
TN 09 AB 1234 => (True, 'TN 09 AB 1234')
TN-09-AB-1234 => (True, 'TN 09 AB 1234')
TN09A12B4 => (False, 'Invalid Format')
T109AB1234 => (False, 'Invalid Format')
TN091234 => (False, 'Invalid Length')
```

Analysis / Discussion

The validator first normalizes the input, then checks length and character types. For TN09AB1234, clean[:2] is TN, clean[2:4] is 09, clean[4:-4] is AB, and clean[-4:] is 1234. This follows the assignment's simplified format; it does not check whether a state code or district code is officially registered.

Result

The registration number validator accepts the specified one- or two-letter series format, normalizes case and separators, and rejects invalid lengths or character patterns.

TASK 4 – Performance Analysis of String Concatenation

Aim

To compare repeated += concatenation with str.join() for building a large string.

Concept / Theory

Python strings are immutable. Repeated concatenation can repeatedly copy existing content and may approach $O(n^2)$ total work. str.join() can determine the final allocation and build the result in O(total output length) time. Exact timings depend on the Python implementation, hardware, and system load.

Procedure / Algorithm

1. Create 200,000 item strings.
2. Build the result using repeated += and measure with time.perf_counter().
3. Build the same result using join() and measure it.
4. Compare equality, output length, and measured time.

Python Program

```
import time

items = [f"Item{i}:Rs{i % 50}" for i in range(200_000)]

def build_plus(parts):
    text = ""
    for part in parts:
        text += part + ";"
    return text

def build_join(parts):
    return ";".join(parts) + ";"

start = time.perf_counter()
a = build_plus(items)
plus_time = time.perf_counter() - start

start = time.perf_counter()
b = build_join(items)
join_time = time.perf_counter() - start

print("Same text:", a == b)
print("Output length:", len(b))
print(f"+= time: {plus_time:.6f} seconds")
print(f"join time: {join_time:.6f} seconds")
print("join is faster:", join_time < plus_time)
```

Sample Output

```
Same text: True
Output length: (computed when the program runs)
+= time: (measured when the program runs)
join time: (measured when the program runs)
join is faster: (depends on the measured run)
```

Analysis / Discussion

The equality check confirms that both approaches build identical text. join() is generally the preferred approach for combining many strings because it avoids repeatedly rebuilding the accumulated string. Runtime values are intentionally printed by the program rather than invented here; run it on the target computer to obtain valid measurements.

Result

The program compares both methods using the same input and verifies correctness. It also reports real execution times and whether join() was faster on that run.

TASK 5 – Functions, Parameters and Scope

Aim

To demonstrate the mutable default-argument problem and correct it using None, while explaining parameter passing and scope.

Concept / Theory

Default argument expressions are evaluated once when a function is defined. Therefore, a mutable default list is shared across calls. Using None and creating a new list inside the function gives each call an independent list. Parameters may be positional, keyword, or default-valued. Local variables exist within a function; global refers to module scope, and nonlocal refers to an enclosing function scope.

Procedure / Algorithm

1. Call a function with a mutable default list more than once and observe shared state.
2. Rewrite it with order=None and create a new list when needed.
3. Store each returned value separately and print the results.
4. Use a caller-provided list to demonstrate intentional list reuse.

Python Program

```
def add_item_buggy(item, order=[]):
    order.append(item)
    return order

def add_item(item, order=None):
    if order is None:
        order = []
    order.append(item)
    return order

# Bug: both calls refer to the same default list.
bug_first = add_item_buggy("Tea")
bug_second = add_item_buggy("Vada")
print("Buggy first:", bug_first)
print("Buggy second:", bug_second)

# Correct: each call gets a fresh list by default.
correct_first = add_item("Tea")
correct_second = add_item("Vada")
print("Correct first:", correct_first)
print("Correct second:", correct_second)

# Passing a list explicitly allows intentional reuse.
customer_order = ["Coffee"]
print("Custom order:", add_item("Vada", customer_order))
```

Sample Output

```
Buggy first: ['Tea', 'Vada']
Buggy second: ['Tea', 'Vada']
Correct first: ['Tea']
Correct second: ['Vada']
Custom order: ['Coffee', 'Vada']
```

Analysis / Discussion

The buggy function shares one list object between calls. The corrected function creates a new list whenever order is None. An explicitly supplied list is modified intentionally. This prevents accidental state sharing between customers.

Result

The mutable default argument bug was demonstrated and corrected using None, ensuring independent default orders while still allowing callers to pass a list when shared modification is intended.

TASK 6 – Recursion and Its Limits

Aim

To compare naive recursive and memoized solutions for counting ways to climb n steps when one or two steps may be taken at a time.

Concept / Theory

The recurrence is f(n) = f(n-1) + f(n-2), with f(0) = f(1) = 1. Naive recursion recomputes the same subproblems and has exponential time complexity. Memoization caches previously computed results and reduces the work to O(n), but recursion still requires O(n) call-stack depth.

Procedure / Algorithm

1. Implement the recurrence using plain recursion and count calls.

2. Implement a cached version using functools.lru_cache.

3. Compare results and call counts for n = 10, 20, and 25.

4. Attempt a sufficiently large n to demonstrate the recursion-depth limitation.

Python Program

```
from functools import lru_cache
import sys

naive_calls = 0

def ways_naive(n):
    global naive_calls
    naive_calls += 1
    if n <= 1:
        return 1
    return ways_naive(n - 1) + ways_naive(n - 2)

memo_calls = 0

@lru_cache(maxsize=None)
def ways_memo(n):
    global memo_calls
    memo_calls += 1
    if n <= 1:
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2)

for n in (10, 20, 25):
    naive_calls = 0
    naive_result = ways_naive(n)
    print(f"Naive n={n}: result={naive_result}, calls={naive_calls}")

    ways_memo.cache_clear()
    memo_calls = 0
    memo_result = ways_memo(n)
    print(f"Memo  n={n}: result={memo_result}, calls={memo_calls}")

try:
    ways_memo(1000)
except RecursionError:
    print("Memoized n=1000: RecursionError (call-stack limit)")
```

Sample Output

```
Naive n=10: result=89, calls=177
Memo  n=10: result=89, calls=11
Naive n=20: result=10946, calls=21891
Memo  n=20: result=10946, calls=21
Naive n=25: result=121393, calls=242785
Memo  n=25: result=121393, calls=26
Memoized n=1000: RecursionError (call-stack limit)
```

Analysis / Discussion

Both methods return the same number of step combinations. The naive version has exponential time complexity, while memoization reduces time to O(n). The memoized version still has O(n) cache storage and O(n) recursion depth; Python may raise RecursionError for deep inputs such as 1000.

Result

Memoization greatly reduces repeated calculations, but it does not remove the recursion call-stack limit. An iterative solution is therefore appropriate for very large n.

TASK 7 – Diagnose, Redesign and Validate

Aim

To remove the recursion bottleneck from step counting by designing an iterative dynamic-programming solution with O(n) time and O(1) auxiliary space.

Concept / Theory

Memoization reduces repeated calculations but retains O(n) recursion depth and cache space. The iterative method computes values from the smallest subproblem upward, keeping only the previous two values. This gives O(n) time and O(1) auxiliary space, excluding the space required to represent the resulting large integer.

Procedure / Algorithm

1. Use the recurrence f(n) = f(n-1) + f(n-2), with f(0) = f(1) = 1.
2. Initialize two variables to represent the previous two results.
3. Update the variables in a loop from 2 through n.
4. Run the function for n = 1000 and report the exact result and its digit count.
5. Measure execution time on the current computer.

Python Program

```
import time

def ways_iterative(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    if n <= 1:
        return 1

    prev2, prev1 = 1, 1
    for _ in range(2, n + 1):
        prev2, prev1 = prev1, prev2 + prev1
    return prev1

start = time.perf_counter()
result_1000 = ways_iterative(1000)
elapsed = time.perf_counter() - start

print("Iterative n=1000 completed successfully.")
print("Exact result:", result_1000)
print("Result digit count:", len(str(result_1000)))
print(f"Execution time: {elapsed:.6f} seconds")
```

Sample Output

```
Iterative n=1000 completed successfully.
Exact result: (the program prints the exact 209-digit integer)
Result digit count: 209
Execution time: (measured when the program runs)
```

Analysis / Discussion

The loop performs n-1 updates, so time complexity is O(n). It stores only two previous values, so auxiliary space is O(1). Python integers can grow to arbitrary precision, so the integer result itself occupies more memory as n increases; this output-size memory is separate from the algorithm's O(1) auxiliary variables. Unlike recursion, this method does not consume one call frame per step.

Result

The recursive bottleneck was redesigned as an iterative solution. It successfully computes the n=1000 result without a recursion error, with O(n) time and O(1) auxiliary space.